Lecture 3: Approximation Theory

or: what functions can we approximate with a neural network?

Speaker: Omar Khattab

adapts slides from Stefanie Jegelka, Phillip Isola, Jeremy Bernstein, François Fleuret

Lecture 3: Approximation Theory

1. Pieces of the ML puzzle: Approximation, Optimization, & Generalization.

2. Approximation Theory: Constructive proof with 3-layer ReLU network.

3. What the theory entails. Limitations & improvements.

4. Width vs. Depth.

Every year, deep nets tackle more and more difficult problems…

Reuters My News

Google clinches milestone gold at global math competition, while OpenAI also claims win

By **Kenrick Cai** and **Jaspreet Singh**

July 22, 2025 6:46 AM EDT · Updated July 22, 2025

msn

OpenAI scores gold in one of the world's top programming competitions

Story by **Andile Sicetsha** • 3w • 2 min read

What can/can't DNNs actually do? Provably?

Hacker News new | past | comments | ask | show | jobs | submit

If you ask CoPilot to solve something it hasn't seen, it won't be able to solve it.

It's a transformer. Do you understand what that means? It's just matrix multiplication.

???

Deep Nets *are* "just" matrix multiplications <u>and some simple non-linearities</u>.
What can or can't be done by just composing these operations?

Recall: The supervised learning problem

Task: We want to solve some prediction problem $\mathbf{g}: \mathbf{X} \rightarrow \mathbf{Y}$.

Challenge: We don't know how to "code" $\mathbf{g}$, like SOLVE_IMO(...).

So, given examples $(\mathbf{x_i}, \mathbf{y_i})$, we seek to learn params $\boldsymbol{\theta}$ to minimize *empirical risk*.

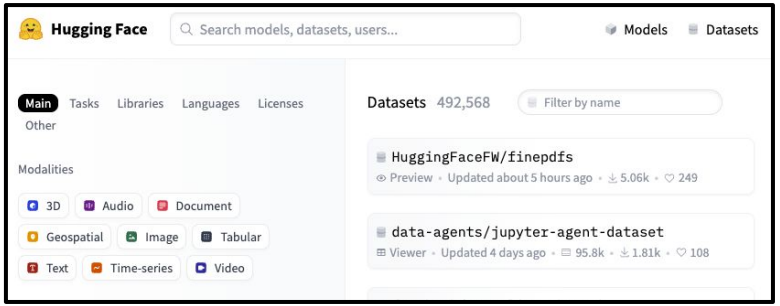
$$\hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_\theta(x_i), y_i\big)$$

But we *hope* to that parameters $\boldsymbol{\theta}$ actually minimize the population risk, i.e. on "unseen" examples from the distribution of inputs.

$$R(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\ell(f_\theta(x), y)\big]$$

What kinds of *problems* are we interested in learning?

Perhaps all datasets on HuggingFace?



Or certain functions from $\text{Image} \rightarrow \text{Classification Label}$, in which the average human annotator can score above 90%?

Or certain functions from Prompt → Natural Language Response?

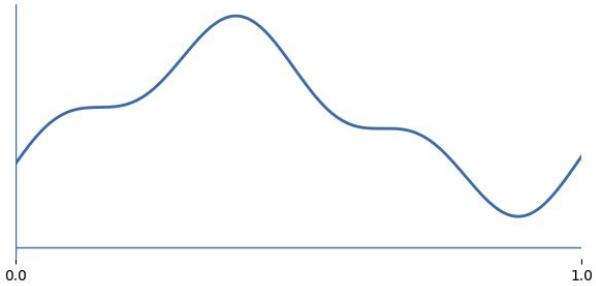
We'll study this question through the lens of <u>regression</u>*

Consider all functions that output a single scalar $f : \mathbb{R}^d \to \mathbb{R}$

Is it *always* possible for our DL methods to express $\mathbf{f}$? To learn $\mathbf{f}$?

What if we also add that $\mathbf{f}$ is continuous? (eliminate pathological functions etc)

Can we approximate $\mathbf{f}$ well with a linear model?



What about a two-layer ReLU network?

*This isn't giving up much. It's not too hard to see that, if we can express a large enough class of regression functions really well, this "buys us" classification and text generations tasks too!

The machine learning puzzle (you need *all* three pieces at once!)

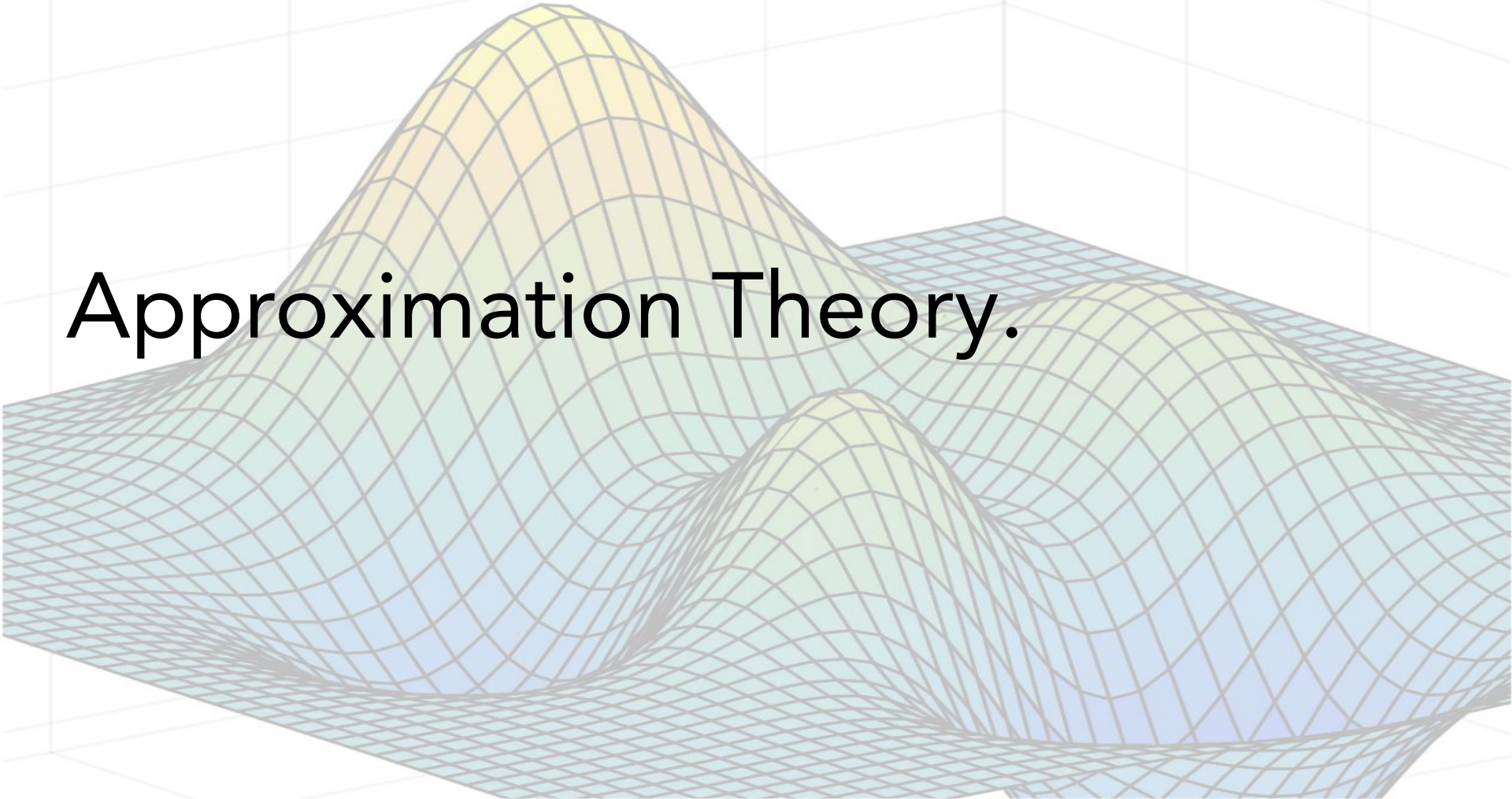
1. Approximation: Does there exist a model in my model family that can actually do my task well?

2. Optimization: If one exists, can I learn it from a bunch of training data?

3. Generalization: Does the model I learned work well on unseen data?

Today, we'll study a little bit of Approximation Theory.

Approximation Theory.



Formalizing the Approximation problem

Given a family of curves $\mathbf{G}$ and a family of neural networks $\mathbf{F}$.

For any curve $\mathbf{g} \in \mathbf{G}$, *is there* a neural net $\mathbf{f} \in \mathbf{F}$ such that $\mathbf{error(f, g) < \varepsilon}$?

Theorem (Universal approximation with 2 layers, *Hornik, Stinchcombe, White 1989; Leshno et al. 1993*):

Suppose we use a continuous activation function that is not a polynomial. Then for any continuous function $g : \mathbb{R}^d \to \mathbb{R}$ and any error $\varepsilon > 0$, there exists a 2-layer network f, with bias terms, such that

$$|f(\mathbf{x}) - g(\mathbf{x})| \leq \varepsilon \quad \text{for all } \mathbf{x} \in [0,1]^d$$

Two-layer MLPs are "universal approximators"! We'll informally prove a weaker result today.

Consider not all continuous functions, but specifically *L*-Lipschitz functions.

Basically, functions **g** that cannot change faster than slope L. *The output difference is bounded by a linear factor of input distance.*

Definition (Lipschitz continuity). A function $g : \mathbb{R}^d \to \mathbb{R}$ is called L-Lipschitz, with constant $L \geq 0$, if

$$|g(\mathbf{x}) - g(\mathbf{x}')| \;\leq\; L\,\|\mathbf{x} - \mathbf{x}'\|$$

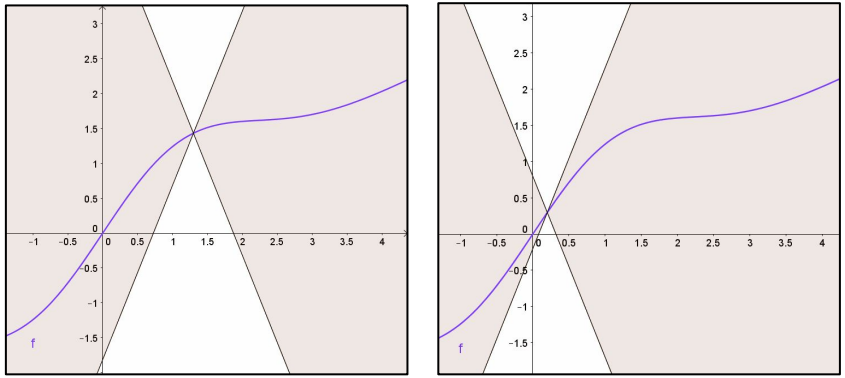
Here, $\|\mathbf{x} - \mathbf{x}'\|$ denotes the *norm* (distance), e.g. ℓ_∞ or Euclidean. We'll use the former today.

$$\|\mathbf{x} - \mathbf{x}'\|_\infty := \max_{1 \leq i \leq d} |x_i - x_i'|.$$

$$\|\mathbf{x} - \mathbf{x}'\|_2 \;=\; \sqrt{\sum_{i=1}^{d} (x_i - x_i')^2}$$

Theorem. Let g be an arbitrary L-Lipschitz function and $\varepsilon > 0$. Then there exists a 3-layer ReLU network f with $\Omega\big((L/\varepsilon)^d\big)$ units such that

$$\int_{[0,1]^d} \big|f(\mathbf{x}) - g(\mathbf{x})\big|\, d\mathbf{x} \;\leq\; 2\varepsilon.$$



https://en.wikipedia.org/wiki/Lipschitz_continuity

To start simple, let's approximate a *univariate* g: R → R first.

With a little bit of iteration, GPT-5 turned a sketch into incomprehensible *matplotlib* magic that produces this visualization :-)

```
def approx(x): # 5 bins
    if   x < 0.2: return 1.02   # ≈ f(0.2)
    elif x < 0.4: return 1.64   # ≈ f(0.4)
    elif x < 0.6: return 0.86   # ≈ f(0.6)
    elif x < 0.8: return 0.48   # ≈ f(0.8)
    else:         return 0.64   # ≈ f(1.0)
```

How many bins (or **`if`** statements) should we create so the error is small?

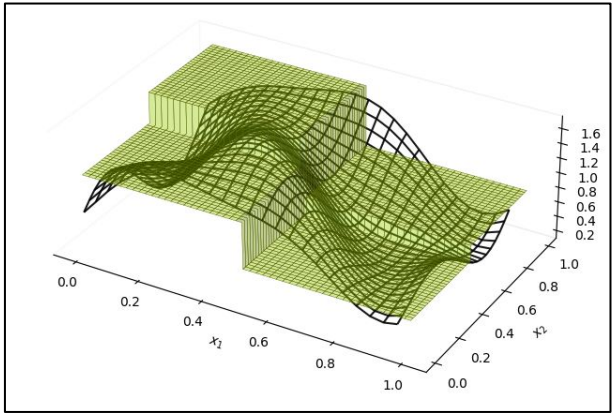
Because g is L-Lipschitz, moving the input by Δx can change g by at most $L|\Delta x|$.

So if each bin has width $\Delta x \leq \varepsilon / L$, then g can change by at most ε within a bin.

Thus, N = L / ε bins can guarantee ε error (at *any* input! and thereby in the integral from 0 to 1 too)

Wait. If a basic Python program with **`if`** *statements does it, why do need MLPs…?*

This works in higher dimensions, but requires *exponentially*-many bins.



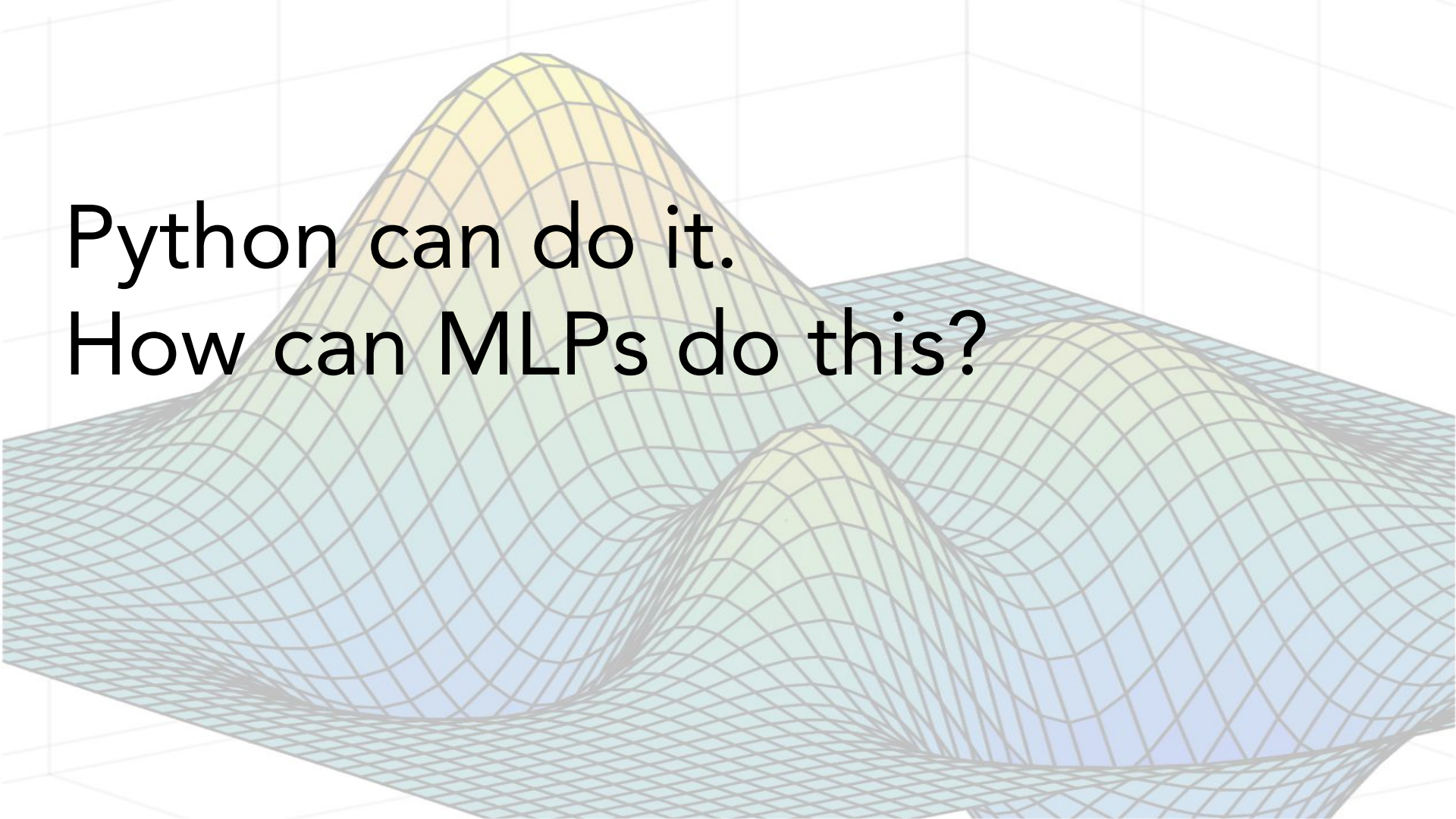
GPT-5 was pretty helpful in turning a sketch to matplotlib code to visualize this.

```
def approx(x1, x2): # 2x2 bins
 if (0 < x1 < 0.5) and (0 < x2 < 0.5): return 1.18
 if (0 < x1 < 0.5) and (0.5 < x2 < 1.0): return 1.60
 if (0.5 < x1 < 1.0) and (0 < x2 < 0.5): return 0.56
 if (0.5 < x1 < 1.0) and (0.5 < x2 < 1.0): return 0.77
```

How many bins (or **`if`** statements) should we create?

Because g is L-Lipschitz, we still want each bin to have side length $\Delta x \leq \varepsilon$ / L in *each* dimension. Using the ℓ_∞ norm, error $|g(\mathbf{x}) - g(\mathbf{x}')| \leq L\|\mathbf{x} - \mathbf{x}'\| \leq L \cdot \epsilon / L = \epsilon$.

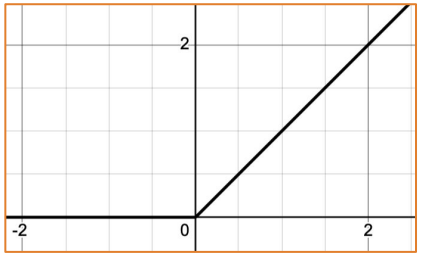
For the step size to be ε / L, <u>N = (L / ε)^d</u> bins are needed overall.



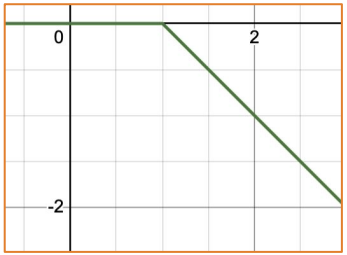
Python can do it.
How can MLPs do this?

Let's prove that a really wide MLP can (approximately!) do the same thing!

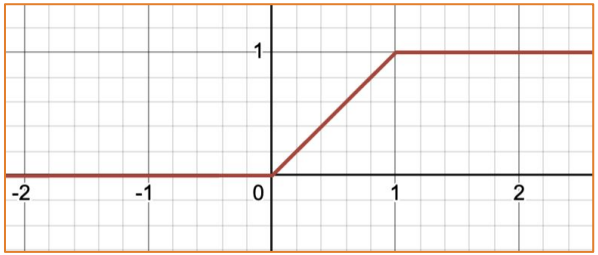
For a single-dimensional input, can we express a conditional like `if u ≤ x < v: return w` with some ReLU compositions? Basically, we want a block of height **w** from **x=u** to **x=v**.



ReLU(x)



-ReLU(x-1)



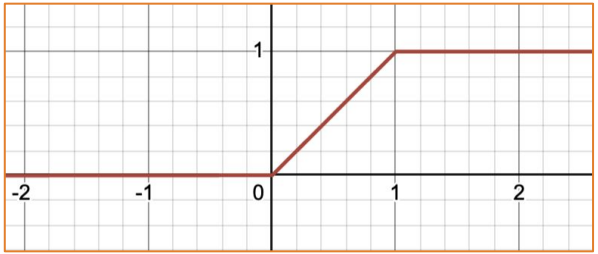
ReLU(x) - ReLU(x-1)

Define U(x, t, g) = ReLU(x - t) - ReLU(x - t - g)

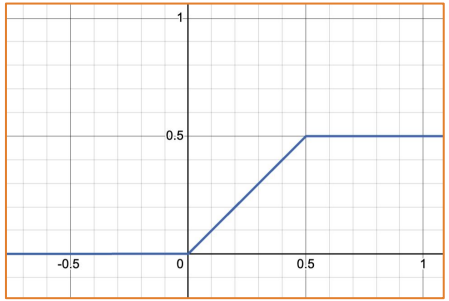
so this above ^ is U(x, 0, 1)

We want a block of height **w** from **x=0** to **x=1**.

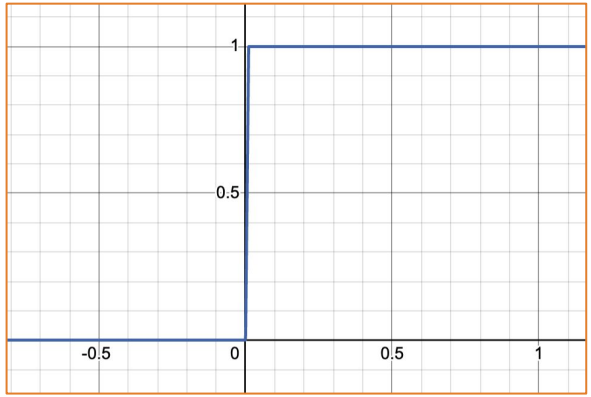
We defined U(x, t, g) = ReLU(x - t) - ReLU(x - t - g).



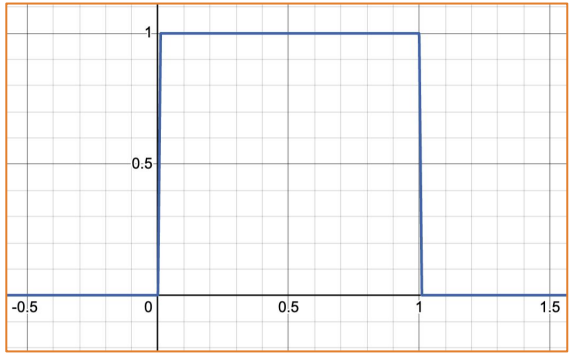
U(x, 0, 1)



U(x, 0, 0.5)



100 * U(x, 0, 0.01)



$\underline{1}/0.01 * [\ U(x, \underline{0}, 0.01) - U(x, \underline{1}, 0.01)]$

We want a ~rectangle of height **w** from any **x=*u*** to **x=*v***.

$$\frac{w}{\gamma}\Big[\mathrm{ReLU}(x-u)-\mathrm{ReLU}(x-u-\gamma)-\mathrm{ReLU}(x-v)+\mathrm{ReLU}(x-v-\gamma)\Big]$$

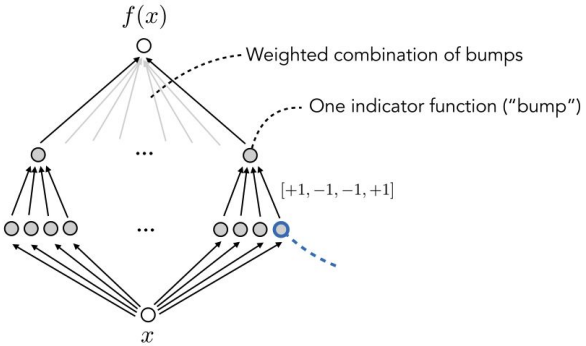
with $\gamma > 0$ being a small constant (e.g., $\gamma = 0.001$).

Equivalently, the 2-layer MLP for each "bump" can be written as:

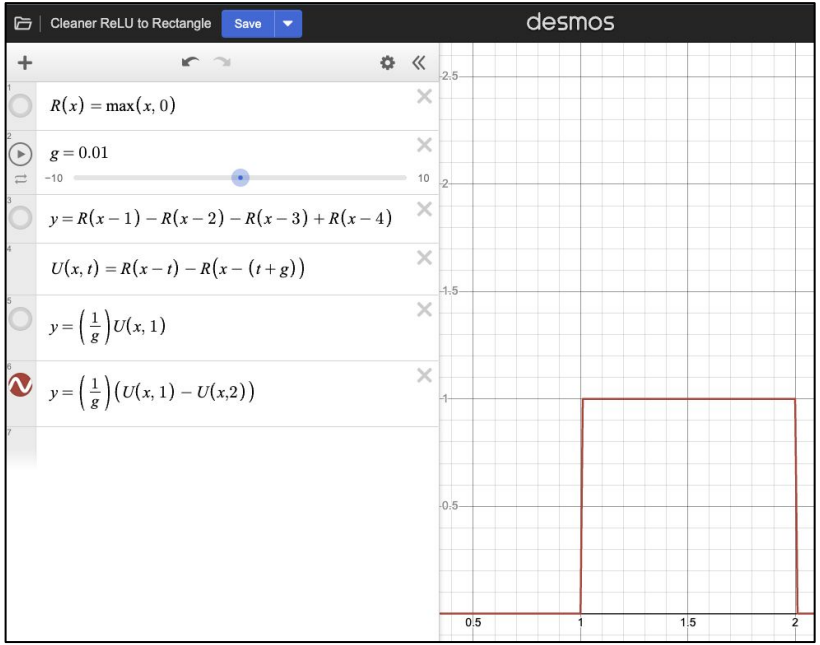
$$\begin{bmatrix}\frac{w}{\gamma} & -\frac{w}{\gamma} & -\frac{w}{\gamma} & \frac{w}{\gamma}\end{bmatrix}\mathrm{ReLU}\left(\begin{bmatrix}1\\1\\1\\1\end{bmatrix}x+\begin{bmatrix}-u\\-(u+\gamma)\\-v\\-(v+\gamma)\end{bmatrix}\right)$$

$f : \mathbb{R} \to \mathbb{R}$

3-Layer ReLU network



Layer 2 and 3 are both strictly linear, hence can collpase into a single linear layer, hence: a 2-layer ReLU net can approx any continuous univariate function arbitrarily well.



Useful graphing website in 2D and 3D at desmos.com (used for the illustrations)

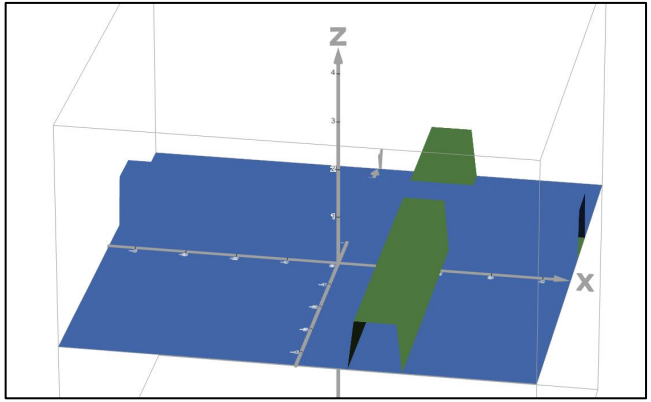
Extending this to multiple dimensions is just addition across dimensions + a threshold at d-1, where they overlap.

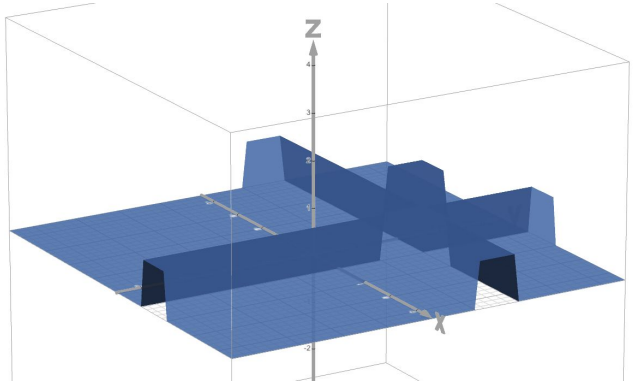
For each dimension x_i, we'll have the same indicator as before:

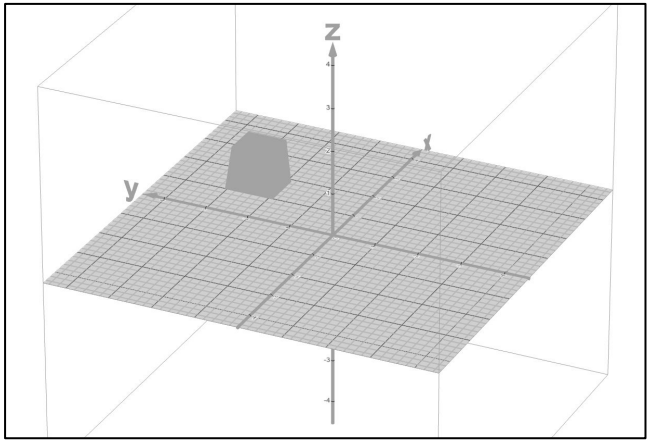
$$f_c(x_i; u_i, v_i) = \frac{w}{\gamma}[\mathrm{ReLU}(x_i - u_i) - ...]$$

And we'll add them up across dimensions to get our "hyper-rectangle":

$$h_c(\mathbf{x}; \mathbf{u}, \mathbf{v}) = \mathrm{ReLU}\left(\sum_{i=1}^{d} f_c(x_i; u_i, v_i) \; - \; (d-1)\, w\right)$$







Proof Summary. And Discussion.

Overall proof structure

Theorem. Let g be an arbitrary L-Lipschitz function and $\varepsilon > 0$. Then there exists a 3-layer ReLU network f with $\Omega\big((L/\varepsilon)^d\big)$ units such that

$$\int_{[0,1]^d} \big|f(\mathbf{x}) - g(\mathbf{x})\big|\, d\mathbf{x} \;\leq\; 2\varepsilon.$$

1. This is easy to show if we had explicit indicator functions (the if statements), starting in one dimension and generalizing to multiple.

   You can understand this Python as:

$$f_{\text{step}}(x) = \sum_j \alpha_j \mathbf{1}[x \in R_j]$$

```
def approx(x): # 5 bins
   if   x < 0.2: return 1.02  # ≈ f(0.2)
   elif x < 0.4: return 1.64  # ≈ f(0.4)
   elif x < 0.6: return 0.86  # ≈ f(0.6)
   elif x < 0.8: return 0.48  # ≈ f(0.8)
   else:         return 0.64  # ≈ f(1.0)
```

2. With some ReLU compositions, it's easy to *approximate* each indicator explicit bump, if our gamma is small enough.

3. *<u>We just then need to add up all the bumps, with one final layer.</u>*

Some limitations of the statement we proved.

- We're talking about width (L / e)^d, i.e. *exponential* in the number of input features! Think about what this means when you have "just" 1000 features.
  - We did NOT prove that it's *required* for DNNs. Only that our proof needs so many!

- We needed very large weights w/gamma. Large weights are often a sign that something is going wrong, e.g. here, it's an explicit lookup-table.

- Limitation of the proof sketch: We didn't account for the approximation error from the fact that gamma is not actually zero. This costs us in the statement: approximation only on average (the integral), not pointwise.

Most important limitation of our statement: If a basic Python program does it, why do need MLPs?

Our proof shows that MLPs *can* be constructed to simulate a lookup table. This is already enough for a very large class of functions (continuous functions).

But that's not what MLPs do in practice and it's not what we want them to do!

In practice, what makes Deep Learning interesting is that *much smaller networks* can learn interesting patterns via SGD, from little data, and they manage to generalize to unseen examples despite that.

This ability to learn highly compressed representations from a little bit of data and to generalize nonetheless are not properties that [at least as far as we know!] arbitrary Python programs have.

tl;dr Being a good approximator of your target function is *necessary* & not sufficient.

Improvements: 2 layers, point-wise

Theorem: Universal approximation with 2 layers *(Hornik, Stinchcombe, White 1989, Leshno et al 1993)*
Suppose we use a continuous activation function that is not a polynomial. Then for any continuous function $g : \mathbb{R}^d \to \mathbb{R}$ and any ϵ, there exists a 2-layer network f with bias terms such that $|f(x) - g(x)| \leq \epsilon$ for all $x \in [0,1]^d$

- uses a "heavy" tool (Stone-Weierstrass theorem), bumps as products
- still exponential in dimension
- other proofs: *Cybenko 1989, Funahashi 1989, Barron 1993*

Improvements: Barron's theorem

- Statement: smooth functions can be approximated with fewer neurons

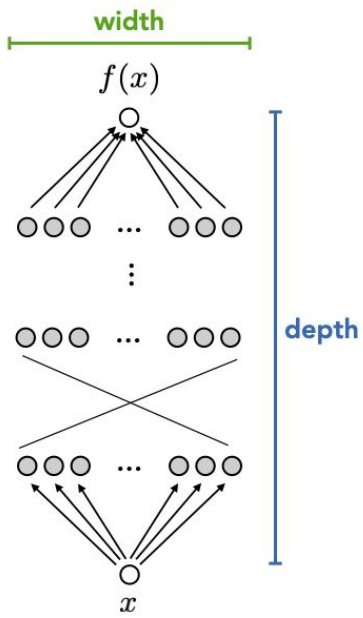
- Construction: use Fourier representation as approximation
  - "Basis functions" are cos functions
  - Approximate with finitely many
  - Approximate cos with other activation functions

Theorem (Barron): For $g : \mathbb{R}^d \to \mathbb{R}$, there exists a 2-layer neural network with k hidden units such that $\int_0^1 |f(x) - g(x)|^2 dx \leq 4C_f/k$ where

$C_f = \|\widehat{\nabla f}\|_1^2 = (\int \|w\| |\hat{f}(w)| dw)^2$

square integral of the Fourier transform of the gradient of f

Should you scale the width or depth of your neural network?



Why might you want to focus on scaling width?

- We just learned that even a 2-layer MLP is a "universal approximator"... if it's wide enough!
- Width is inherently parallelizable. Depth is sequential.
- Width is easier to train. Depth can lead to gradient problems.

So… scaling width is obviously better?

Why might you want to focus on scaling *depth*?

- It works better in practice!
- It makes sense intuitively for feature processing?
- And.. "*depth separation*" results!

It's possible to describe classes of functions that

- don't require a lot of neurons if you stack them deeply
- but require exponentially more units to fit with shallow networks!

We'll describe two: Piecewise linear functions & Sawtooth functions.

Why might you want to focus on scaling *depth*?

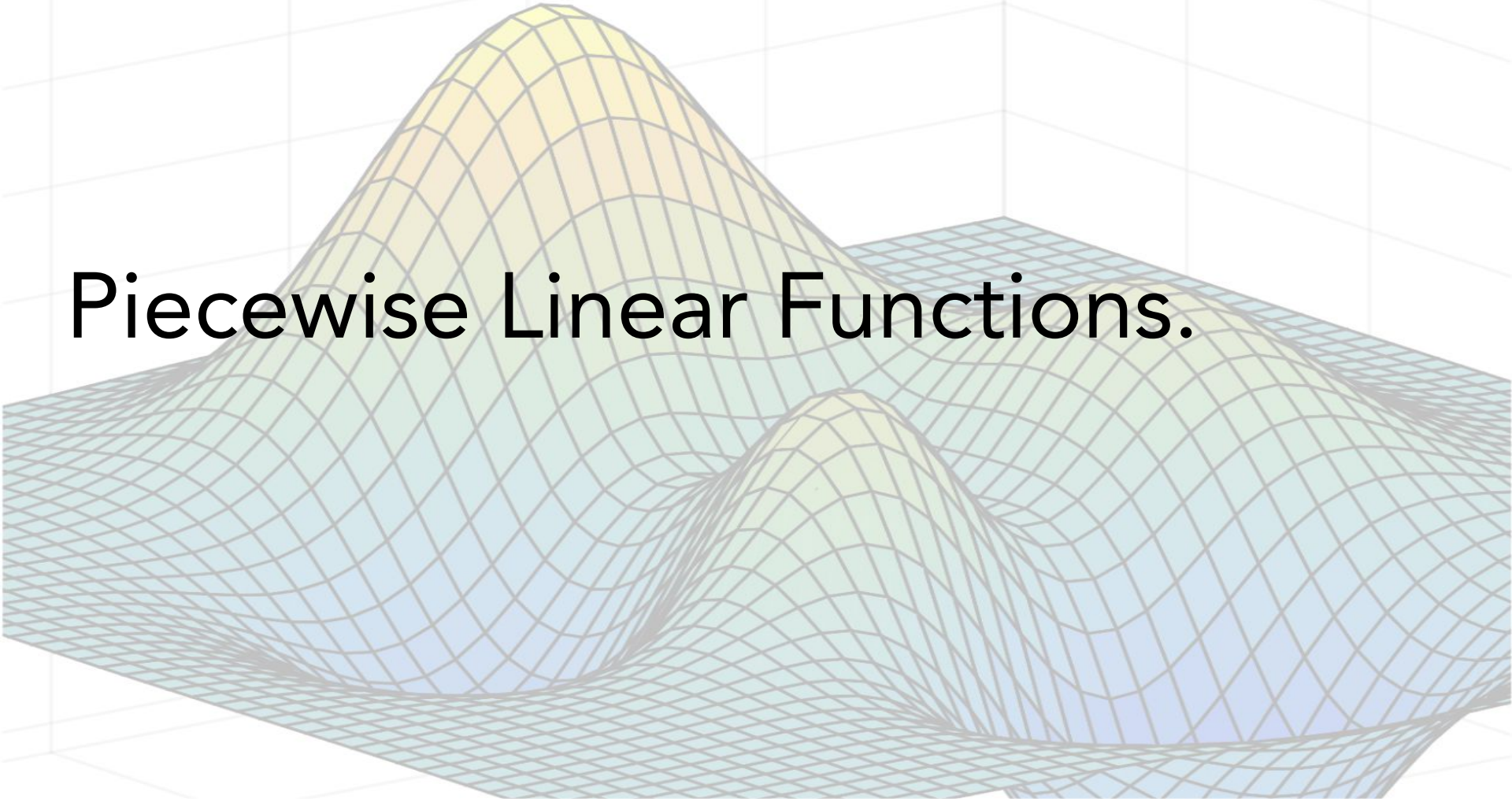
- "Depth separation" results:

It's possible to describe classes of functions that

- don't require a lot of neurons if you stack them deeply
- but require exponentially more units to fit with shallow networks!

We'll describe two: Piecewise linear functions & Sawtooth functions.

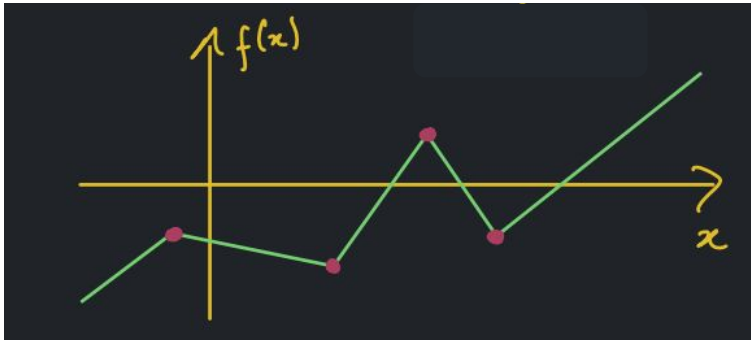
Piecewise Linear Functions.



Piecewise linear ("PWL") functions

Define a *kink* to be a place where the gradient changes.

Here, we have 4 kinks.



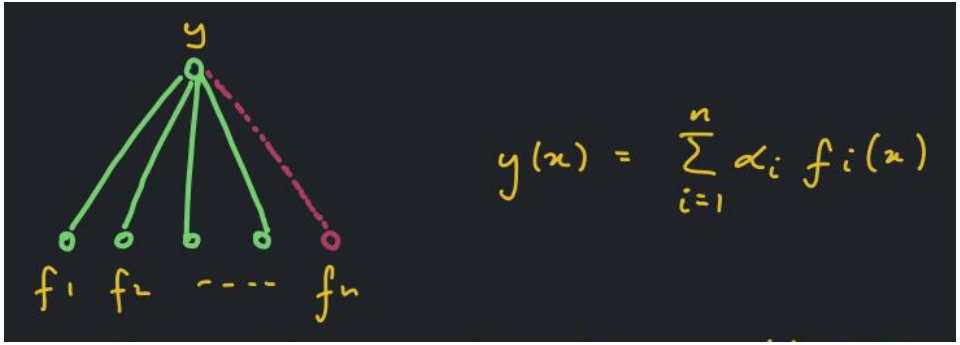
Claim: ReLU networks are PWL. Why?

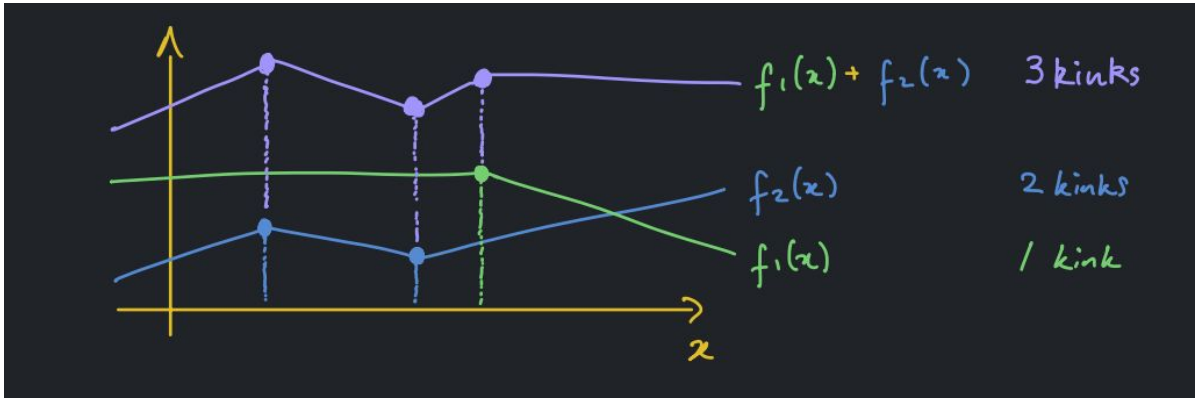
1. ReLU is PWL. ---/
2. f PWL ⇒ α f is PWL for any real number α
3. f, g both PWL ⇒ f + g is PWL
4. f, g both PWL ⇒ f ∘ g is PWL

Depth separation result based on number of “kinks”!

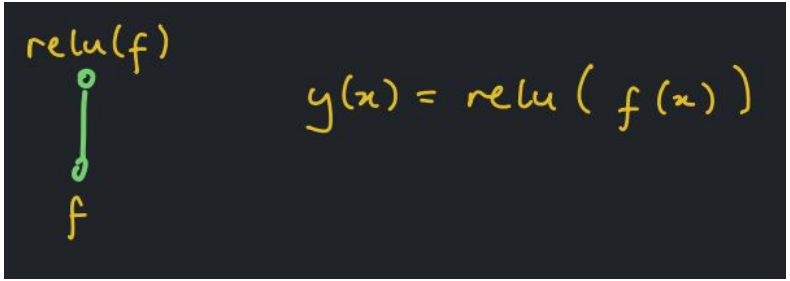
Effect of adding width.

When we add functions, we at most *add* the number of kinks.

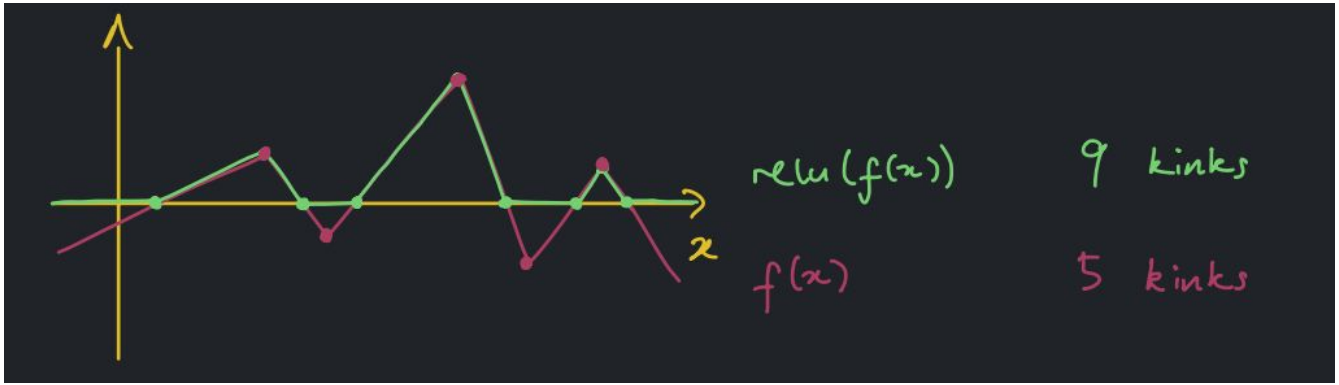


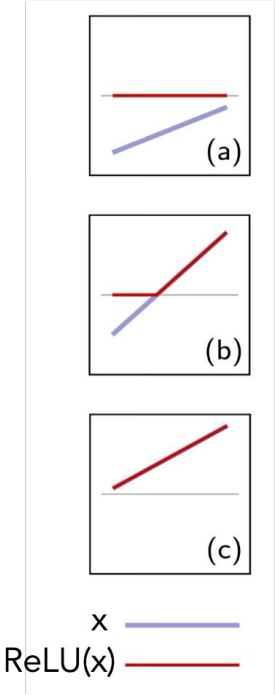


Effect of adding depth.



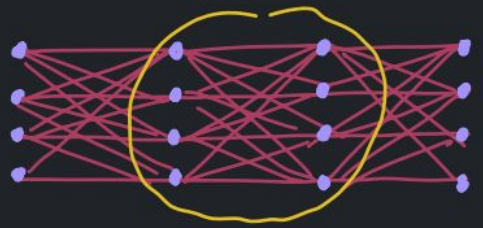
Every time we apply a ReLU(.), we at most *double* the number of kins.





Bonus Slide

More formally



Consider the ℓth layer:

vectors in $\mathbb{R}^n$

$$f_\ell(x) = \mathrm{relu}\left(W_\ell \, f_{\ell-1}(x) + b_\ell \right)$$

vector in $\mathbb{R}^n$

$n \times n$ matrix

Let KINKS_ℓ denote the max number of kinks over the n coordinates of $f_\ell(x)$.

Then it holds that $\mathrm{KINKS}_\ell \leq 2n \cdot \mathrm{KINKS}_{\ell-1}$

Since $\mathrm{KINKS}_0 = 1$, this implies $\boxed{\mathrm{KINKS}_L \leq (2n)^L}$

The number of kinks in the function at layer L

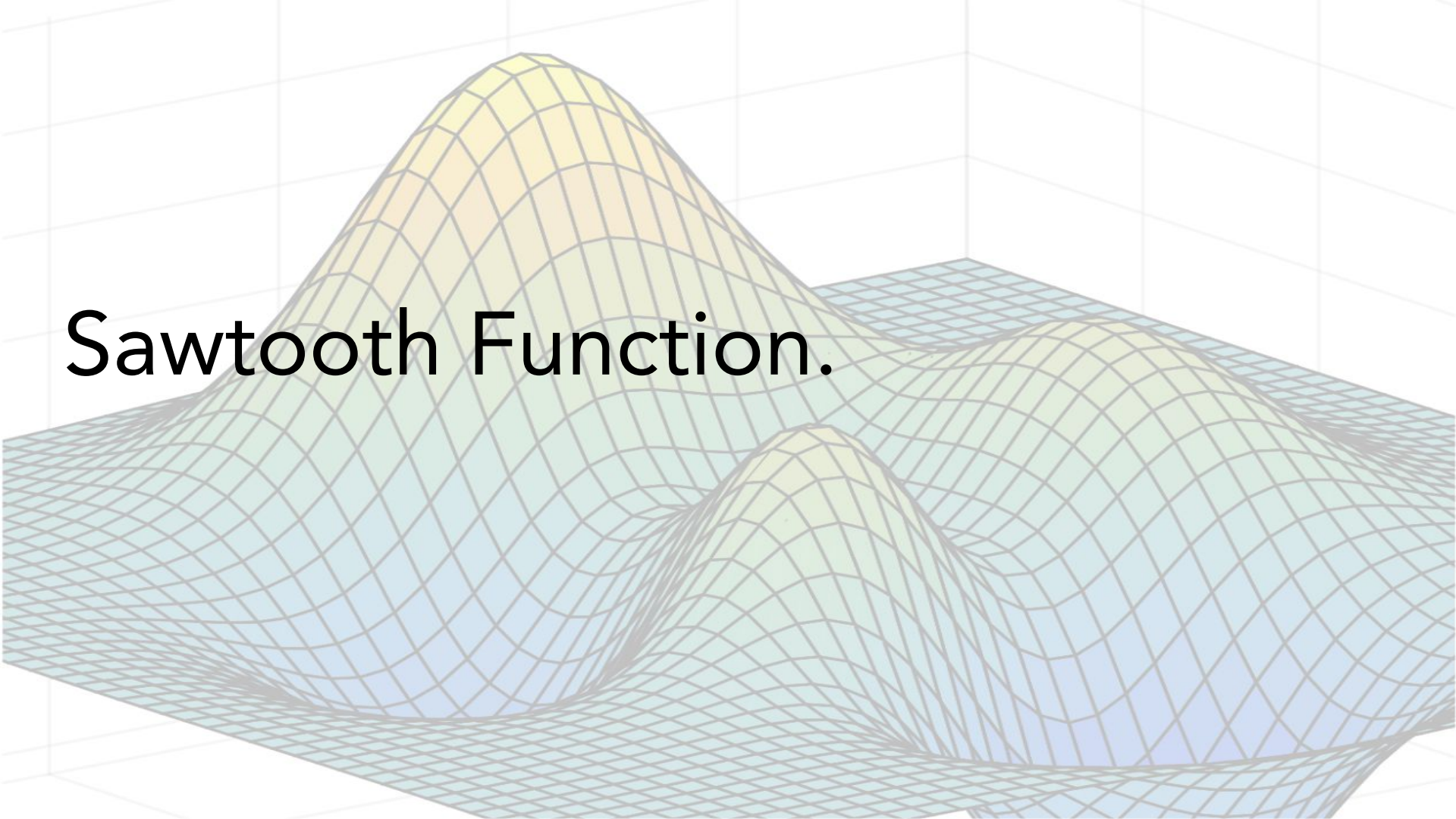
Define Kinks_L = # kinks in the function at layer L

$\text{Kinks}_L \leq (2n)^L$ where n = width and L = depth

At most, this upper bound grows polynomially in width but exponentially in depth.

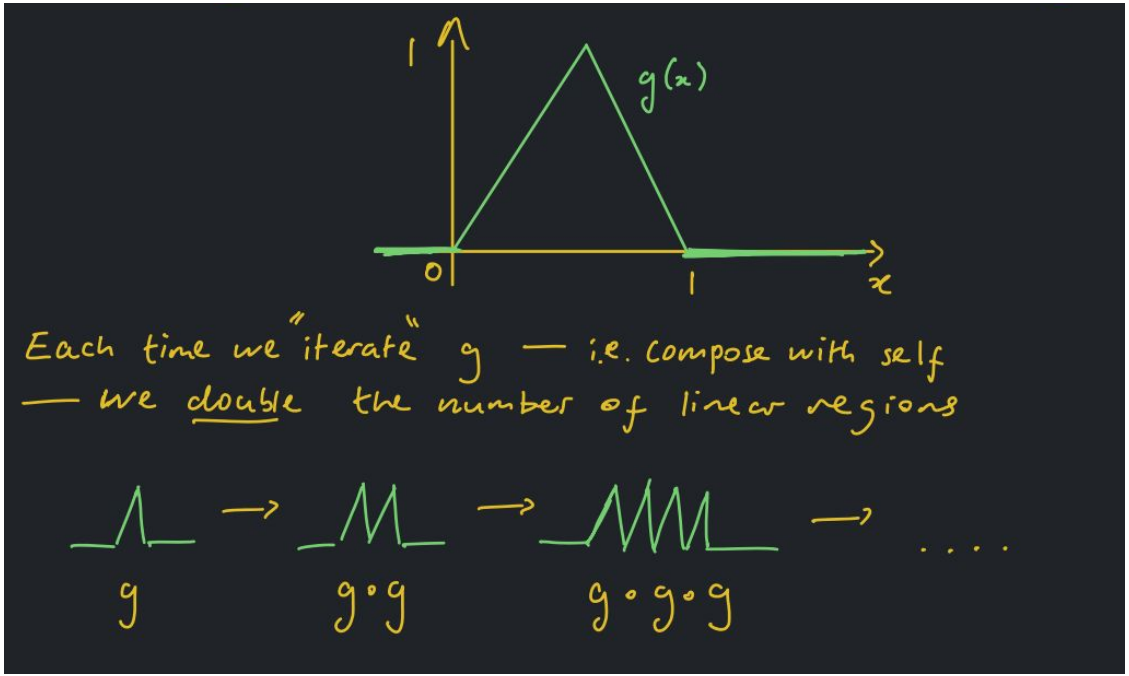
Need to ask: Is this upper bound ever attained?

Sawtooth Function.



Answer: Yes, with the sawtooth function.

$$g(x) = \texttt{relu}(2\texttt{relu}(x) - 4\texttt{relu}(x - \frac{1}{2}))$$



Depth separation: sawtooth function

Theorem *(Telgarsky 2015,2016)*: For any $L > 1$ there exists a function $g(x)$ such that $g(x)$ can be expressed by a (deep) ReLU network with $3L^2 + 6$ units and $2L^2 + 4$ layers, but any (shallow) ReLU network with L layers and $\leq 2^L$ nodes cannot approximate it to error $\|f - g\|_1 < 1/32$.

Note: You can approximate it with a 2-layer network, but need exponentially more units

Depth Separation for the sawtooth function.

$g \circ g \circ \cdots \circ g$ (500 times) has $2^{500} - 1$ kinks

It has 500 layers, with width 2.

To get the same number of kinks, a 3-layer network would need width n in…

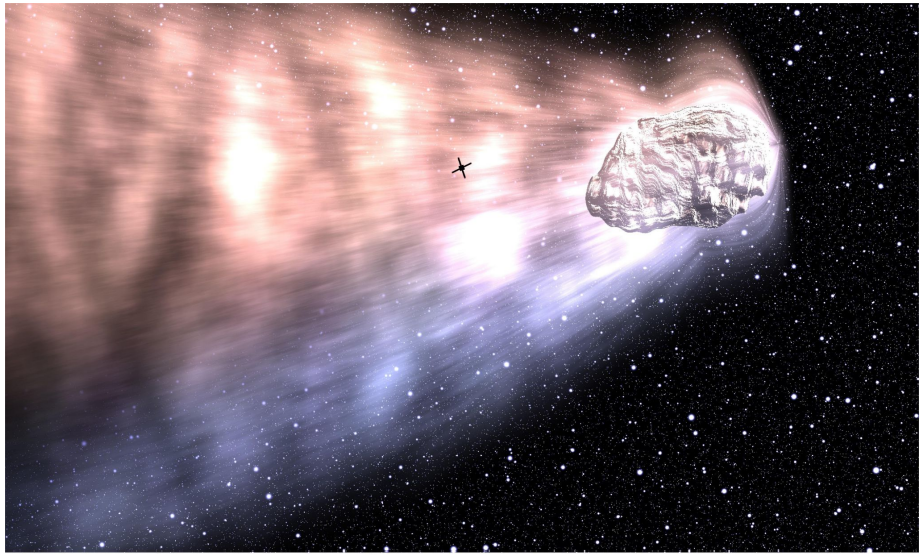
$$\text{Kinks} \leq (2n)^L$$

$$\Rightarrow n \geq \tfrac{1}{2}\, \text{Kinks}^{(1/L)}$$

$$= \tfrac{1}{2}\,(2^{500} - 1)^{1/3}$$

$$= 7 \times 10^{49} \text{ units}$$

Maybe after this lecture, it's less (or perhaps more?) surprising that *very short* compositions of simple mathematical expressions can draw pretty sophisticated art…



Comet by Hamid Naderi Yeganeh

The above picture is a converted version of a 2000×1200 image. For $m = 1, 2, 3, \ldots, 2000$ and $n = 1, 2, 3, \ldots, 1200$, the color of the pixel of the row n and the column m is

$$\mathrm{rgb}\left(F\left(H_0\left(\frac{m-1000}{600}, \frac{601-n}{600}\right)\right), F\left(H_1\left(\frac{m-1000}{600}, \frac{601-n}{600}\right)\right), F\left(H_2\left(\frac{m-1000}{600}, \frac{601-n}{600}\right)\right)\right), \text{ where } F(x) = \left\lfloor 255 e^{-e^{-1000x}} |x|^{e^{-e^{1000(x-1)}}} \right\rfloor, \text{ and}$$

$$H_v(x,y) = \frac{20+v^2-5v}{20} C_1(x,y) C_0(x,y) e^{-e^{1000(A_1(x,y)C_1(x,y)-A_0(x,y)C_0(x,y))}} + \frac{5v^2-5v+10}{2}\left(1-T(x,y)\right)\left(1-C_0(x,y)\right)e^{-e^{-}}$$

$$+O(x,y)\left(1-N(x,y)\right)\left(4-2C_1(x,y)\right)e^{-e^{-\frac{3}{10}\left(x+\frac{y}{4}-\frac{7}{10}\right)}} W_v(x,y), \qquad O(x,y) = 1 - e^{-e^{200\sqrt{25x^2+(5y-2)^2}-40}}\left(1-\right.$$

$$T(x,y) = \prod_{s=1}^{40} e^{-e^{500\left(\cos^4\left(4\left(\frac{29}{25}\right)^s\left(\cos(17s^2)x+\sin(17s^2)y\right)+2\cos(19s^2)\right)\cos^4\left(4\left(\frac{29}{25}\right)^s\left(\cos(17s^2)y-\sin(17s^2)x\right)+2\cos(21s^2)\right)-1\right)}}, \quad W_v(x,y) = \sum_{s=1}^{40} \frac{9^s}{10^s} \frac{M_s(x,y}{100}$$

$$A_v(x,y) = 3\sqrt{\left|1-\left(Q_v(x,y)\right)^2-\left(P_v(x,y)\right)^2\right|} + 2R_v(x,y), \qquad G_{v,s}(x,y) = (100+10v^2-35v)e^{-e^{-3\left(y-\frac{x}{4}-\frac{1}{10}\right)}} + (10v^2-5v+70)$$

$$N(x,y) = e^{-e^{5000\left|x+\frac{y}{5}-\frac{2}{15}\right|-50} - e^{5000\left|y-\frac{2}{5}-\frac{x}{5}\right|-50}}, \qquad M_s(x,y) = e^{-e^{-\frac{3}{2}\left(\cos^2\left((B(x,y))^{-1}s^2 4^{-s}\left(y-\frac{x}{4}-\frac{1}{5}\right)+\frac{3}{2}\cos\left((5+2\cos(2s^2))\left(x+\cos(12s^2)\frac{y}{5}\right)+s^2\right)+2\cos(5s^2)\right)\cos\right.}}$$

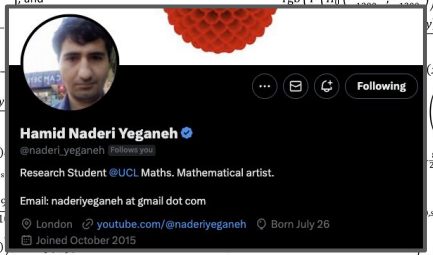
$$R_v(x,y) = \sum_{s=1}^{40}\left(\frac{81}{100}\right)^s e^{-e^{5-10\cos\left(\left(\frac{6}{5}\right)^s K_{v,s}(x,y)+2\cos(2s^2)\right)\cos\left(\left(\frac{6}{5}\right)^s L_{v,s}(x,y)+2\cos(3s^2)\right)}}, \quad B(x,y) = \frac{5}{4}\left(\frac{1}{2}+\frac{1}{\pi}\arctan\left(15x+\frac{15y}{4}-\frac{39}{2}\right)\right) - \frac{7}{20} - \frac{2}{5}\left|x+\frac{y}{4}-1\right.$$

$$K_{v,s}(x,y) = \arcsin\left(\left(\cos(s^2)\,Q_v(x,y)+\sin(s^2)\,P_v(x,y)\right)C_v(x,y)\right), \quad L_{v,s}(x,y) = \arcsin\left(\left(\cos(s^2)\,P_v(x,y)-\sin(s^2)\,Q_v(x,y)\right)C_v(x,y)\right)$$

$$P_v(x,y) = V_v(x,y) + 4^{-1}\cos\left(2V_v(x,y)+2^{-1}U_v(x,y)+2\cos\left(4^{-1}U_v(x,y)+4^{-1}V_v(x,y)\right)\right) + 10^{-1}\cos\left(4V_v(x,y)-2U_v(x,y)+1\right)+10^{-1}3\cos\left(12V_v(x,y)+7U_v(x,y)\right),$$

$$Q_v(x,y) = 10^{-1}7U_v(x,y)+4^{-1}\cos\left(U_v(x,y)-V_v(x,y)+2\cos\left(2^{-1}U_v(x,y)-V_v(x,y)\right)\right)+10^{-1}\sin\left(3U_v(x,y)+2V_v(x,y)+1\right)+10^{-1}3\cos\left(18U_v(x,y)-4V_v(x,y)\right),$$

$$U_v(x,y) = 5x+4^{-1}5y-2^{-1}9+1000^{-1}v, \qquad V_v(x,y) = 5y-4^{-1}5x-1+2000^{-1}v.$$



Polar Bear Walking in Front of Sunset by Hamid Naderi Yeganeh

The above picture is a converted version of a 2000×1200 image. For $m = 1, 2, 3, \ldots, 2000$ and $n = 1, 2, 3, \ldots, 1200$, the color of the pixel of the row n and the column m is

$$\mathrm{rgb}\left(F\left(H_0\left(\frac{m-1000}{1200}, \frac{601-n}{1200}\right)\right), F\left(H_1\left(\frac{m-1000}{1200}, \frac{601-n}{1200}\right)\right), F\left(H_2\left(\frac{m-1000}{1200}, \frac{601-n}{1200}\right)\right)\right), \text{ where } F(x) = \left\lfloor 255 e^{-e^{-1000x}} |x|^{e^{-e^{1000(x-1)}}} \right\rfloor, \text{ and}$$

$$\ldots v)\left(\left(200A_{300}(x,y)+20(7-5v^2+10v)A_{75}(x,y)+(20+3v^2-15v)\left(1-A_{300}(x,y)\right)\left(11-4\sqrt{x^2+y^2}\right)\right)Z_{30}(x,y)+200R_v(x,y)\right)$$

$$\ldots(x,y), \qquad C(x,y) = 1-e^{-e^{12\left(B(x,y)+W(x,y)\right)}-e^{-400\left(y+\frac{1}{250}\cos(7x+50y)+\frac{2}{5}+\frac{E(x,y)}{500}\right)}}, \qquad A_v(x,y) = e^{-e^{v\left(\frac{19}{20}x^2+y^2+\frac{1}{2000}\cos(47x+14y+\cos(28x-9y))-\frac{1}{20}\right)}},$$

$$\ldots\left(\frac{7}{20}\left(10\left(x-\frac{1}{500}\right)-\frac{63}{100}\right)^2+7\left|10\left(y+\frac{8}{25}\right)-\frac{1}{10}\right|^{\frac{5}{2}-6\left(x-\frac{1}{500}\right)}+\frac{8}{25}\sin^8\left(10x-\frac{1}{50}+\frac{2}{5}\right)-\frac{1}{10}\sin^{100}\left(10\left(x-\frac{1}{500}\right)+\frac{4}{5}\right)-\frac{28}{25}+\frac{E(x,y)}{30}\right)\right)$$

$$\ldots\frac{8}{25}\right)-\frac{43}{20}\right)+\frac{1}{2}, \qquad W(x,y) = \sum_{s=1}^{4} 10e^{-e^{-125+500\left(10\left(y+\frac{8}{25}\right)+\frac{1}{2}\right)^2+\frac{E(x,y)}{30}-300\left(\frac{1}{10}+100\left(y+\frac{8}{25}\right)^2\right)}\left(10x-\frac{5}{2}+\frac{13}{10}+\frac{1}{10}\left(\frac{9}{10}+\frac{23}{10}(-1)^s\right)\left(10\left(y+\frac{8}{25}\right)+\frac{2}{5}\right)+\frac{1}{10}\cos^{20}\left(10\left(y+\frac{8}{25}\right)+\frac{7}{10}\right)\right)^2},$$

$$\ldots s(x,y)\frac{3v^2-15v+20}{20}\left(\frac{5-(v-1)^2}{2}-\frac{9-2(v-1)^2}{4}J_{20,s}(x,y)\right)\left(\frac{3}{2}-x^2-y^2\right), \quad Z_s(x,y) = \prod_{u=0}^{s} 1-J_{50,u}(x,y), \quad E(x,y) = \sum_{s=1}^{50}\frac{25^s}{26^s}T_s(x,y),$$

$$J_{v,s}(x,y) = e^{-e^{-100\left(s-\frac{1}{2}\right)-e^{v\left(\left(x+\frac{7}{10}\cos(9s^2)+\frac{1}{10}\cos(27x+50y+\cos(18x-20y)+5s)\right)^2+40\left(y-\frac{1}{10}-\frac{2}{5}\cos(s^2)+\frac{1}{100}\cos(25x-15y+\cos(9x+7y)+7s)\right)^2-\frac{1}{10}+\frac{E(x,y)}{30}\right)}}},$$

$$T_s(x,y) = \cos\left(5^{-1}5^s 4^{-s}(1+\cos(10s))(\sin(s^2)\,y-\cos(s^2)\,x+2\cos(17s))+4\cos\left(10^{-1}5^s 4^{-s}(\sin(9s)\,y-\cos(9s)\,x)\right)+2\cos(5s)\right)$$

$$\times\cos\left(5^{-1}5^s 4^{-s}(1+\cos(10s))(\cos(s^2)\,y+\sin(s^2)\,x+2\cos(15s))+4\cos\left(10^{-1}5^s 4^{-s}(\sin(8s)\,y-\cos(8s)\,x)\right)+2\cos(7s)\right).$$

Summary and comments

- Main idea: approximate a function by a sum of “basis” functions
- Tells us fundamental limitations, with practical implications
- In practice, can seemingly do with much smaller networks…
- Shallow networks cannot easily approximate deep networks (efficiency)
- Building blocks for understanding approximation with other architectures